UC RIVERSIDE

Sunwoo Jang <sjang047@ucr.edu>

[Sunwoo Jang] Brief Update

cong.liu@ucr.edu <cong.liu@ucr.edu> Mon, Sep 28, 2026 at 6:52 AM
To: Seonwoo Jang <seonwoo.jang@email.ucr.edu>
Cc: Soheil Shirvani <sshir030@ucr.edu>

Hi Sunwoo,

Take a look at SpaceROS (https://space.ros.org/) and the following discussion summary I had with AI. I think it would be fantastic if the paper topic we discussed in this email thread gets directly motivated by starship. So basically if you follow the RTAS best paper and what Soheil has been doing, and find a unique motivation/story to connect OS with SpaceROS, that would be a novel contribution I think.

****

Starship's strongest motivation for a better OS is maintaining timely, correct control while hardware fails, mission phases change, and ground support becomes unavailable. Public evidence supports that research direction, but does **not** establish that Starship's present OS is inadequate.

I checked primary sources available as of September 28, 2026. The distinction between SpaceX's disclosed practices and Starship's undisclosed current configuration matters here.

What SpaceX has publicly disclosed:

Area	What the evidence establishes
OS foundation	In 2020, SpaceX engineers described a custom Linux build with **PREEMPT_RT**, relatively few kernel modifications, and custom hardware drivers. Other onboard computers were microcontrollers running bare-metal code. This is historical SpaceX-wide evidence, not a current Starship configuration specification.
Real-time engineering	Engineers described carefully assigned process/kernel-thread priorities, avoiding runtime allocation and unbounded loops, and telemetry checking deadlines across flight phases. Much control software was single-threaded to reduce synchronization nondeterminism.
Fault tolerance	Multiple computers execute and vote on results. A failed computer can reboot and rejoin. Engineers explicitly placed fault management and computing redundancy in **custom application-level software**.
Starship architecture	Starship's software lead described distributed control of engines, flaps, radios, and other subsystems, with some control centralized and some placed near the hardware. Starship reused ideas and some code from Falcon.
Current public evidence	Current Starship job descriptions cover application software, OS, networking, firmware, simulation, real-time distributed systems, and fault tolerance. They do not disclose the deployed kernel version, scheduling policies, or redundancy topology.

The first three rows come directly from SpaceX's 2020 software-team AMA; the Starship-specific statements are from its 2021 AMA. Current hiring materials corroborate the breadth of the stack, rather than its precise implementation. reddit.com

I could not verify Starship's current kernel release, processor configuration, worst-case timing bounds, or use of GPUs/ML in flight-critical control. Those should remain unknowns in any research motivation.

For comparison, the relevant state of the art spans several layers:

Approach	What it already provides	Remaining limitation relevant to Starship
Real-time Linux	PREEMPT_RT reduces nonpreemptible execution; SCHED_DEADLINE supplies EDF/CBS CPU reservations with admission control.	CPU scheduling mechanisms alone do not establish a sensor-to-actuator deadline across networks, devices, and fault recovery. The Linux Kernel documentation
Space RTOSes: VxWorks, RTEMS	Established real-time foundations. ESA's RTEMS SMP qualification package now includes release 6.2, Category B.	Qualification applies to selected configurations and still requires mission-specific work. The RTEMS configuration described by ESA uses a flat memory model without user/kernel protection. NASA
Separation kernels: PikeOS; high-	Partitioning, controlled communication, and stronger isolation foundations. seL4's MCS	These do not automatically prove an entire spacecraft safe. seL4's published verification

Approach	What it already provides	Remaining limitation relevant to Starship
assurance microkernels: seL4	configuration adds temporal resource control.	scope has configuration-specific exclusions, and MCS proofs remain in progress. SYSGO
NASA cFS and F′	Reusable flight-software components, communication, commands, telemetry, and OS abstraction.	They are **frameworks above an OS**. Mission timing, recovery policy, and control correctness still require engineering. Goddard Engineering and Technology Directorate
DLR ScOSA research	Distributed spacecraft computing with reconfiguration after node failures. A 2025 paper implements online task placement to avoid storing exponentially many predetermined configurations.	Adaptive recovery already has substantial prior work. A new contribution needs stronger timing, consistency, or resource guarantees. elib.dlr.de

There is also an active **Space Grade Linux** effort under ELISA, involving NASA and industry. Its stated goal is a common distribution and ecosystem for space missions; that is distinct from a universally qualified spacecraft OS. ELISA

I would prioritize the following eight challenges. **These are mission-driven gaps to investigate, not confirmed defects in Starship.** Some require cooperation between the kernel, middleware, flight applications, and hardware.

1. **Fault recovery with a bounded interruption to control**

   Radiation can corrupt data, disrupt execution, or damage hardware. SpaceX already describes voting and reboot/reintegration; the harder question is **whether useful control remains timely throughout recovery**. NASA

   Recovery itself consumes CPU, memory bandwidth, and network capacity. Restarting a replica may interfere with the surviving controllers precisely when redundancy is reduced.

   OS opportunity: reserve resources for recovery, isolate corrupted components, bound failover latency, and admit a recovered replica only after validating its state. Analyze normal execution and recovery together under an explicit fault model.

2. **End-to-end deadlines and sensor-data freshness**

   Starship's distributed control architecture makes local thread deadlines only part of the problem. A control result can arrive on time yet use an old sensor sample or inconsistent timestamps. reddit.com

   The relevant chain includes acquisition, transport, estimation, planning/control, and actuator delivery.

   OS opportunity: propagate deadlines and timestamps through IPC and network paths; coordinate CPU and communication reservations; discard obsolete work. The guarantee should cover **response time and state age**, including during failover.

3. **Isolation across shared hardware resources**

   If flight control, diagnostics, communications, and richer autonomy share a multicore platform, CPU priority alone cannot control interference through caches, DRAM, DMA, interrupts, or accelerators. Research comparing partitioning hypervisors explicitly identifies remaining real-time isolation challenges. arxiv.org

   OS opportunity: coordinate CPU budgets with memory-bandwidth and I/O isolation, and contain driver failures.

   GPU scheduling becomes relevant if future autonomy uses accelerators. DLR is investigating GPU integration into ScOSA, but that does **not** establish current Starship GPU use. elib.dlr.de

4. **Safe changes between mission modes**

   Launch, coast, docking, propellant transfer, and landing activate different dependencies and resource demands. NASA's Starship propellant-management work specifically includes pressure control, settling, transfer, and chill-down operations. Project

   My inference: the difficult scheduling interval may be the transition, when old work remains queued while new critical work begins.

   OS opportunity: prevalidated mode configurations, bounded transition protocols, atomic budget changes, and cancellation of obsolete work. An application should not enter a new mode before its required computing and communication resources are available.

5. **Autonomous recovery from incorrect behavior**

   For Mars missions, Earth–Mars communication takes roughly **3–22 minutes one way**. Ground intervention cannot resolve an urgent control failure. NASA Science

Watchdogs detect missing progress, but healthy-looking software can produce incorrect commands. A NASA study of 55 selected historical aerospace software incidents found that **85% involved erroneous output rather than silence**. Identical replicas can also agree on the same software error. NASA Technical Reports Server (NTRS)

OS/runtime opportunity: independently protected monitors, guaranteed resources for fallback controllers, and controlled degradation. Mission engineers must supply the safety conditions; the OS must ensure their monitors and fallbacks remain executable.

6. **Distributed control authority during docking and refueling**

   This has a concrete Starship-specific basis: NASA's July 2026 plan says the Starship test article will control the **combined docked Starship–Orion spacecraft** during its portion of the demonstration. NASA

   My inference: failures during handover create questions beyond ordinary task scheduling: Which computer has authority? Can a rebooted node issue an old command? Do both vehicles agree on the active mode?

   OS/middleware opportunity: explicit authority epochs, rejection of stale commands, bounded communication, and recovery protocols that prevent conflicting actuator ownership. Physical actions require application-level semantics as well.

7. **Scheduling when available computing capacity changes**

   A failed node, power restriction, or thermal limit can reduce effective capacity. ScOSA's node-failure reconfiguration already addresses part of this problem. elib.dlr.de

   My inference: a schedule validated for nominal hardware cannot simply be assumed valid after capacity changes.

   OS opportunity: maintain feasible resource allocations for defined degraded configurations; preserve mandatory control services; reduce optional processing in a predictable order. Resource management should coordinate with vehicle power and thermal management rather than allowing performance changes to invalidate timing assumptions silently.

8. **Long-duration updates and recovery without losing consistency**

   SpaceX already described primary/backup update schemes for Starlink that tolerate interrupted updates and recover known-good software. reddit.com

   The harder extension is updating a redundant distributed spacecraft while preserving compatible state, timing assumptions, and sufficient healthy replicas. Restoring an executable alone does not establish that its saved configuration or estimator state is valid.

   OS/runtime opportunity: transactional software/configuration updates, version-compatible state migration, protected recovery images, and persistent diagnostic evidence—with resource limits that keep maintenance from disrupting control.

On Tue, Jul 28, 2026 at 3:23 PM Seonwoo Jang <seonwoo.jang@email.ucr.edu> wrote:

> Hello Cong,
>
> Thank you for sharing! I'm currently digesting AGX, FluidOS outline, and the Linux kernel scheduler.
>
> My rough understanding from your observation is that the upper layer's (ROS2 dispatching) mechanism makes kernel-level (lower layer) scheduling optimization useless. So we might further dive into this gap.
>
> I will carefully read the RTAS'25 paper and think about possible kernel-side directions. I'm also excited about the opportunity to collaborate with the RWTH Aachen group.
>
> Best regards,
> Sunwoo
>
> On Tue, Jul 28, 2026 at 5:07 PM <cong.liu@ucr.edu> wrote:
>
> > Hi Sunwoo,
> >
> > I am visiting Jian-Jia Chen's group at RWTH Aachen in Germany and discussing potential collaborative paper topics. One interesting idea is to have a coordinated middleware (ROS 2) + kernel approach for resource scheduling and management aiming at robotic settings. There is a best paper award at RTAS'25 that targets this problem in a broad sense. Please take a look at the attached.
> >
> > Regarding ROS 2, the students at RWTH (e.g., Harun Teper) who will collaborate with us on this topic are ROS experts so we do not need to worry too much on the middleware part, we can focus on the kernel. It would be great if we can really accomplish such a collaboration and submit a paper to RTSS or Eurosys next summer.